МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
Кафедра інформаційних технологій

# ЛАБОРАТОРНА РОБОТА №5

З дисципліни: «Об’єктно-орієнтоване та подійно-орієнтоване програмування»
На тему: «Особливості та призначення інтерфейсів та принципу успадкування»

Роботу виконав:
студент II курсу групи АД-251
спеціальності F6
Горбунова А.А.

Роботу перевірив:
к т.н., доцент Рудніченко М.Д.

Оцінка: _____

ДУОП, 2026

# 1 МЕТА РОБОТИ ТА ЗАВДАННЯ

## 1.1 Мета лабораторної роботи

Метою лабораторної роботи є отримати практичні навички зі створення і використання абстрактних класів та інтерфейсів, засвоїти принципи успадкування класів у мові програмування Java, навчитися створювати ієрархії класів, перевизначати методи базових класів, реалізовувати методи інтерфейсів та використовувати поліморфізм під час роботи з об’єктами. У процесі виконання роботи необхідно:

## 1.2 Загальне завдання (Практична частина)

Під час виконання лабораторної роботи необхідно створити Java-програму, у якій буде продемонстровано використання успадкування, абстрактного класу, інтерфейсу та поліморфізму.:

1) Створення базового абстрактного класу;
2) Створення дочірніх класів;
3) Створення інтерфейсу;
4) Створення об’єктів;
5) Демонстрація поліморфізму;
6) Робота з інтерфейсом;
7) Розширення програми: Після виконання базової частини необхідно адаптувати приклад до індивідуального завдання. Програма повинна містити:

а) щонайменше один абстрактний клас;
б) щонайменше два дочірні класи;
в) щонайменше один інтерфейс;
г) реалізацію інтерфейсу конкретним класом;
г) перевизначення методів;
д) конструктори базового та дочірніх класів;
е) використання `super`;

є) демонстрацію поліморфізму;

ж) створення та використання об’єктів у методі `main()`.

### 1.3 Індивідуальне завдання. Варіант №7. Комп’ютерна техніка.

1) Створити абстрактний клас `ComputerDevice` з полями `brand` та `model`;
2) Створити класи `Laptop` та `Desktop`;
3) Додати інтерфейс `Connectable` з методом `connect()`.

### 1.4 Оформити звіт і відповісти на контрольні питання.

# 2 ОПИС СТВОРЕННЯ КОДУ ЗАГАЛЬНОГО ЗАВДАННЯ

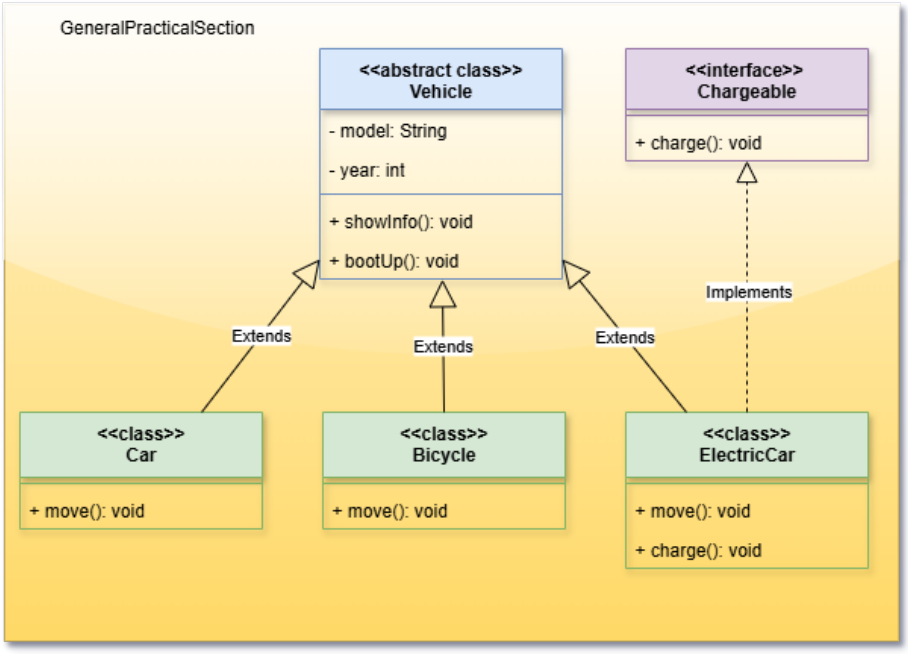



Рисунок 2.1 – UML діаграма класів загального завдання лабораторної роботи №5

# 3 ЛІСТИНГ ПРОГРАМНОГО КОДУ ЗАГАЛЬНОГО ЗАВДАННЯ

У цьому розділі наведено повний вихідний коду загального завдання. Код оформлено згідно зі стандартами *Oracle Code Conventions* та містить вичерпну документацію у форматі *JavaDoc*.

## 3.1 Клас Vehicle.java

```
1  /*
2   * Copyright (c) 2026. Anastasiia Gorbunova
3   * National University "Odesa Polytechnic"
4   *
5   * Licensed under the Apache License, Version 2.0 (the "License");
6   * you may not use this file except in compliance with the License.
7   * You may obtain a copy of the License at
8   *
9   *     http://apache.org
10  */
11 package ad251.gorbunova;
12
13 /**
14  * Базовий абстрактний клас для секції загальної практики (GeneralPracticalSection).
15  * Надає базову структуру для всіх транспортних засобів.
16  *
17  * @author Анастасія Горбунова
18  * @version 5.0 23 Oct 2026
19  */
20 public abstract class Vehicle {
21     /** Модель транспортного засобу */
22     protected String model;
23     /** Рік випуску */
24     protected int year;
25
26     /**
27      * Конструктор для ініціалізації загальних характеристик транспортного засобу.
28      *
29      * @param model марка або модель
30      * @param year рік виробництва
31      */
32     public Vehicle(String model, int year) {
33         this.model = model;
34         this.year = year;
35     }
36
37     /**
38      * Абстрактний метод руху, що повертає текстовий статус поточного стану.
39      *
40      * @return лаконічний опис дії транспортного засобу
41      */
42     public abstract String move();
43
44     // --- Публічні геттери ---
45     public String getModel() { return model; }
46     public int getYear() { return year; }
47 }
```

## 3.2 Інтерфейс Chargeable.java

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Інтерфейс для пристроїв, що підтримують заряджання.
 *
 * @author Анастасія Горбунова
 * @version 5.0 15 Oct 2026
 */
public interface Chargeable {
    void charge();
}
```

## 3.3 Клас Car.java

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Дочірній клас Car, що розширює Vehicle (Extends).
 * Представляє класичний автомобіль.
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public class Car extends Vehicle {
    /**
     * Конструктор класу Car. Передає параметри базовому класу.
     *
     * @param model марка авто
     * @param year рік випуску
     */
    public Car(String model, int year) {
        super(model, year);
    }

    /**
     * Перевизначений метод руху автомобіля.
     *
     * @return рядок із описом руху автомобіля
     */
    @Override
    public String move() {
        return "Рухається загальною дорогою.";
    }
}

```

## 3.4 Клас Bicycle.java

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Дочірній клас Bicycle, що розширює Vehicle (Extends).
 * Представляє велосипед.
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public class Bicycle extends Vehicle {
    /**
     * Конструктор класу Bicycle. Передає параметри базовому класу.
     *
     * @param model марка велосипеда
     * @param year рік випуску
     */
    public Bicycle(String model, int year) {
        super(model, year);
    }

    /**
     * Перевизначений метод руху велосипеда.
     *
     * @return рядок із описом руху велосипедною доріжкою
     */
    @Override
    public String move() {
        return "Рухається велосипедною доріжкою.";
    }
}

```

## 3.5 Клас ElectricCar.java

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Клас Електромобіль. Поєднує успадкування Vehicle (extends)
 * та реалізацію Chargeable (implements).
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public class ElectricCar extends Vehicle implements Chargeable {
    /**
     * Конструктор класу ElectricCar. Передає параметри базовому класу.
     *
     * @param model модель електромобіля
     * @param year рік випуску
     */
    public ElectricCar(String model, int year) {
        super(model, year);
    }

    /**
     * Перевизначений метод руху електромобіля.
     *
     * @return рядок із описом руху через електродвигун
     */
    @Override
    public String move() {
        return "Рухається за допомогою електродвигуна.";
    }

    /**
     * Реалізація методу інтерфейсу Chargeable.
     * Виконує контракт інтерфейсу без зміни повертаємого типу.
     */
    @Override
    public void charge() {
        // Контрактний метод інтерфейсу
    }
}
```

# 4 ОПИС СТВОРЕННЯ КОДУ ІНДИВІДУАЛЬНОГО ЗАВДАННЯ

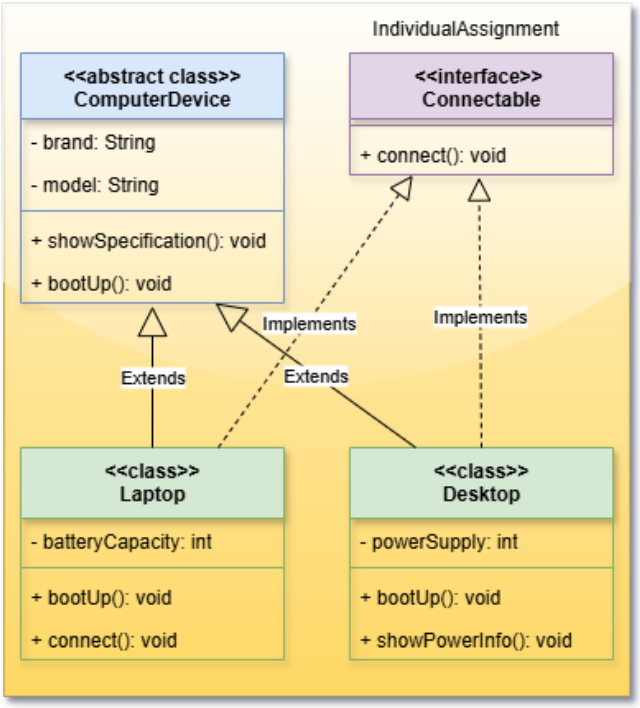



Рисунок 4.1 – UML діаграма класів індивідуального завдання лабораторної роботи №5

# 5 ЛІСТИНГ ПРОГРАМНОГО КОДУ ІНДИВІДУАЛЬНОГО ЗАВДАННЯ

У цьому розділі наведено повний вихідний код індивідуального завдання. Варіант №7. Код оформлено згідно зі стандартами *Oracle Code Conventions* та містить вичерпну документацію у форматі *JavaDoc*.

## 5.1 Клас ComputerDevice.java

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Базовий абстрактний клас, що описує обчислювальну комп'ютерну техніку
 * для секції індивідуального завдання. Варіант 7. (IndividualAssignment).
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public abstract class ComputerDevice {
    /** Торгова марка (бренд) пристрою */
    protected String brand;
    /** Конкретна модель пристрою */
    protected String model;

    /**
     * Конструктор абстрактного класу для ініціалізації базових полів.
     *
     * @param brand бренд виробника
     * @param model назва моделі
     */
    public ComputerDevice(String brand, String model) {
        this.brand = brand;
        this.model = model;
    }

    /**
     * Абстрактний метод для ініціалізації завантаження операційної системи.
     *
     * @return лаконічний статус завантаження ОС у вигляді рядка
     */
    public abstract String bootUp();

    // --- Публічні геттери ---
    public String getBrand() { return brand; }
    public String getModel() { return model; }
}
```

## 5.2 Інтерфейс Connectable.java

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Інтерфейс, що визначає поведінку пристроїв, здатних підключатися до мережі.
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public interface Connectable {
    /**
     * Абстрактний метод для встановлення мережевого з'єднання.
     */
    void connect();

    /**
     * Повертає тип мережевого інтерфейсу пристрою.
     *
     * @return назва типу підключення (Wi-Fi, Ethernet тощо)
     */
    String getNetworkType();
}
```

## 5.3 Клас Laptop.java

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Дочірній клас Laptop. Поєднує успадкування ComputerDevice (extends)
 * та реалізує інтерфейс Connectable (implements).
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public class Laptop extends ComputerDevice implements Connectable {
    /** Ємність вбудованої акумуляторної батареї у мАг */
    private final int batteryCapacity;

    /**
     * Конструктор класу Laptop. Викликає конструктор суперкласу через super().
     *
     * @param brand бренд пристрою
     * @param model модель ноутбука
     * @param batteryCapacity ємність батареї
     */
    public Laptop(String brand, String model, int batteryCapacity) {
        super(brand, model);
        this.batteryCapacity = batteryCapacity;
    }

    /**
     * Переозначений метод завантаження ноутбука.
     *
     * @return скорочений статус швидкого старту
     */
    @Override
    public String bootUp() {
        return "Швидкий запуск (Режим сну)";
    }

    /**
     * Реалізація методу інтерфейсу Connectable для підключення.
     */
    @Override
    public void connect() {
        // Логіка підключення об'єкта (внутрішній стан)
    }

    /**
     * Повертає бездротовий тип мережі ноутбука.
     *
     * @return рядок "Wi-Fi"
     */
    @Override
    public String getNetworkType() {
        return "Wi-Fi";
    }

    public int getBatteryCapacity() {
        return batteryCapacity;
    }
}
```

## 5.4 Клас Desktop.java

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Дочірній клас Desktop. Поєднує успадкування ComputerDevice (extends)
 * та реалізує інтерфейс Connectable (implements).
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public class Desktop extends ComputerDevice implements Connectable {
    /** Потужність блоку живлення в Ватах */
    private final int powerSupply;

    /**
     * Конструктор класу Desktop. Передає параметри базовому класу.
     *
     * @param brand бренд пристрою
     * @param model модель системного блоку
     * @param powerSupply потужність БП
     */
    public Desktop(String brand, String model, int powerSupply) {
        super(brand, model);
        this.powerSupply = powerSupply;
    }

    /**
     * Перевизначений метод завантаження стаціонарного ПК.
     *
     * @return скорочений статус повного старту
     */
    @Override
    public String bootUp() {
        return "Повне завантаження (Перевірка живлення)";
    }

    /**
     * Реалізація методу інтерфейсу Connectable для кабельного підключення.
     */
    @Override
    public void connect() {
        // Логіка дротового підключення об'єкта
    }

    /**
     * Повертає дротовий тип мережі десктопа.
     *
     * @return рядок "Ethernet"
     */
    @Override
    public String getNetworkType() {
        return "Ethernet";
    }

    public int getPowerSupply() {
        return powerSupply;
    }
}
```

# 6 ПЛАНУВАННЯ ВИВЕДЕННЯ РЕЗУЛЬТАТІВ ВИКОНАННЯ ЗАВДАНЬ В ФАЙЛІ MAIN.JAVA

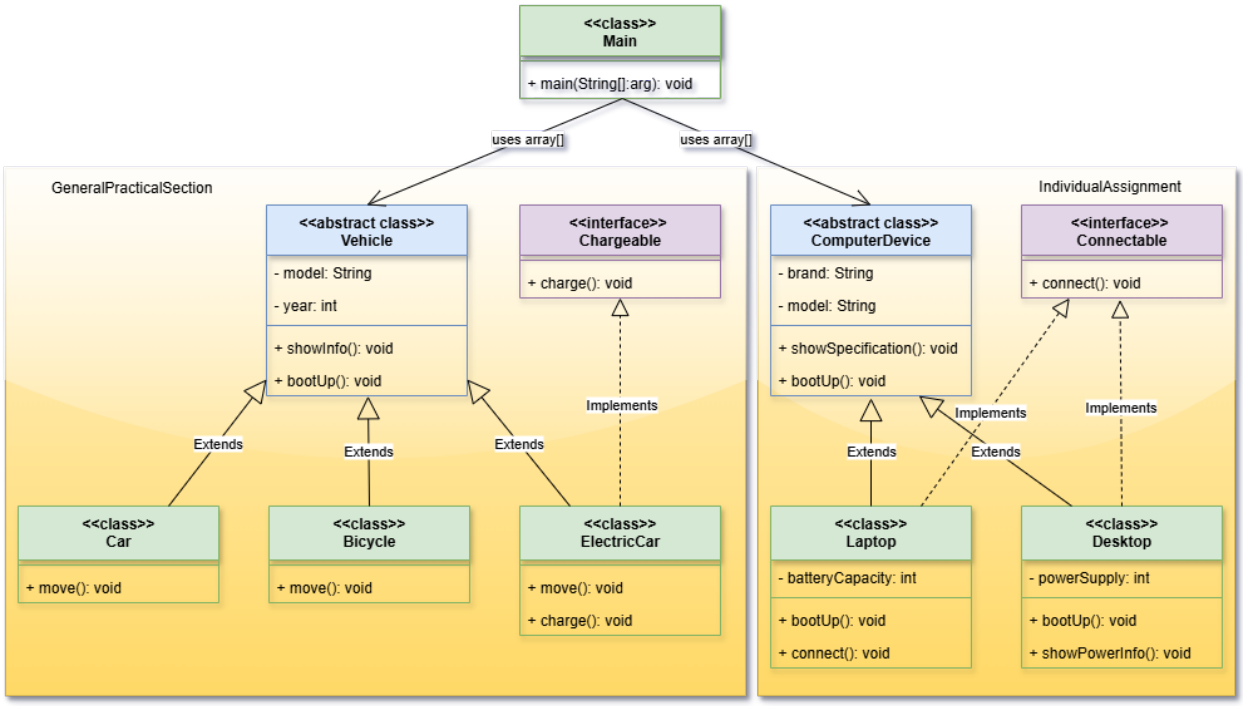



Рисунок 6.1 – UML діаграма взаємодії головного файлу Main.java з класами загального та індивідуального завдань лабораторної роботи №5

# 7 ЛІСТИНГ ПРОГРАМНОГО КОДУ ТОЧКИ ВХОДУ

У цьому розділі наведено повний вихідний код. Код оформлено згідно зі стандартами *Oracle Code Conventions* та містить вичерпну документацію у форматі *JavaDoc*.

## 7.1 Клас Main.java

```
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Головний керуючий клас програми (Точка входу).
 * Координує виконання та форматування результатів у вигляді таблиць CLI.
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public class Main {

    /**
     * Точка входу в систему. Формує структурований псевдотабличний лог.
     *
     * @param args аргументи командного рядка CLI
     */
    public static void main(String[] args) {
        System.out.print("=== Лабораторна робота №5 ===\n");
        System.out.print("Тема: Особливості призначення інтерфейсів та принципу
        ↪  успадкування.\n\n");

        // ======================================================================
        // 1. ЗАГАЛЬНА ПРАКТИЧНА ЧАСТИНА (Транспортні засоби)
        // ======================================================================
        System.out.print("--- 1. Демонстрація загального прикладу (Транспортні засоби) ---\n");
        String vSeparator =
        ↪  "+------------+--------------+------+----------+-------------------------------------------
```

```
        Vehicle[] vehicles = {
            new Car("Toyota", 2022),
            new Bicycle("Trek", 2023),
            new ElectricCar("Tesla", 2024)
        };

        System.out.println(vSeparator);
        System.out.printf("| %-10s | %-12s | %-4s | %-8s | %-53s |%n", "Тип", "Марка/Модель",
        ↪  "Рік", "Зарядка", "Поточний стан / Дія");
        System.out.println(vSeparator);

        for (Vehicle v : vehicles) {
            String type = "Автомобіль";
            if (v instanceof Bicycle) type = "Велосипед";
            if (v instanceof ElectricCar) type = "Електрокар";

            // Безопасная проверка интерфейса без вызова void-метода в строку
            String chargeStatus = (v instanceof Chargeable) ? "+" : "-";

            System.out.printf("| %-10s | %-12s | %-4d | %-8s | %-53s |%n",
                    type, v.getModel(), v.getYear(), "   " + chargeStatus, v.move());
        }
        System.out.println(vSeparator);
        System.out.print("\n");

```

```
61          // ========================================================================
62          // 2. ІНДИВІДУАЛЬНЕ ЗАВДАННЯ: ВАРІАНТ №7 (Комп'ютерна техніка)
63          // ========================================================================
64          System.out.print("--- 2. Демонстрація індивідуального Варіанту №7 (Комп'ютерна техніка)
            ↪  ---\n");
65          String dSeparator =
            ↪  "+------------+------------+----------------+-----------+-------------------------------
66
67          ComputerDevice[] devices = {
68              new Laptop("Lenovo", "ThinkPad T470s", 4600),
69              new Desktop("ASUS", "ROG Strix", 750),
70              new Laptop("Apple", "MacBook Pro", 5800)
71          };
72
73          System.out.println(dSeparator);
74          System.out.printf("| %-10s | %-11s | %-15s | %-10s | %-41s |%n", "Тип", "Виробник",
            ↪  "Модель", "Мережа", "Статус ОС (bootUp)");
75          System.out.println(dSeparator);
76
77          for (ComputerDevice cd : devices) {
78              String typeLabel = (cd instanceof Laptop) ? "Ноутбук" : "Десктоп";
79              String netLabel = " ";
80
81              if (cd instanceof Connectable) {
82                  netLabel = ((Connectable) cd).getNetworkType();
83              }
84
85              System.out.printf("| %-10s | %-11s | %-15s | %-10s | %-41s |%n",
86                      typeLabel, cd.getBrand(), cd.getModel(), netLabel, cd.bootUp());
87          }
88
89          // Демонстрація поліморфізму через інтерфейсне посилання
90          Connectable officeLaptop = new Laptop("HP", "EliteBook", 5000);
91          if (officeLaptop instanceof ComputerDevice) {
92              ComputerDevice polyDevice = (ComputerDevice) officeLaptop;
93              System.out.printf("| %-10s | %-11s | %-15s | %-10s | %-41s |%n",
94                      "* Ноутбук", polyDevice.getBrand(), polyDevice.getModel(),
                        ↪  officeLaptop.getNetworkType(), polyDevice.bootUp());
95          }
96
97          System.out.println(dSeparator);
98          System.out.println("[АННОТАЦІЯ] * — Об'єкт створено через інтерфейсний тип посилання
            ↪  (Connectable dev = new Laptop(...)).");
99          System.out.println("                Демонстрація динамічного поліморфізму та доступу
            ↪  виключно до методів інтерфейсу.");
100     }
101 }
```

# 8 РЕЗУЛЬТАТИ ВИКОНАННЯ РОБОТИ

Для підтвердження коректності роботи розробленої ієрархії класів, перевірки працездатності абстрактних методів та демонстрації динамічного поліморфізму через інтерфейсні типи посилань було виконано компіляцію та запуск коду. Компіляція та виконання програми здійснювалися в терміналі PowerShell середовища розробки VS Code з примусовим встановленням кодування UTF-8.

На рисунку **??** представлено скріншот вікна терміналу з результатами виконання програми.

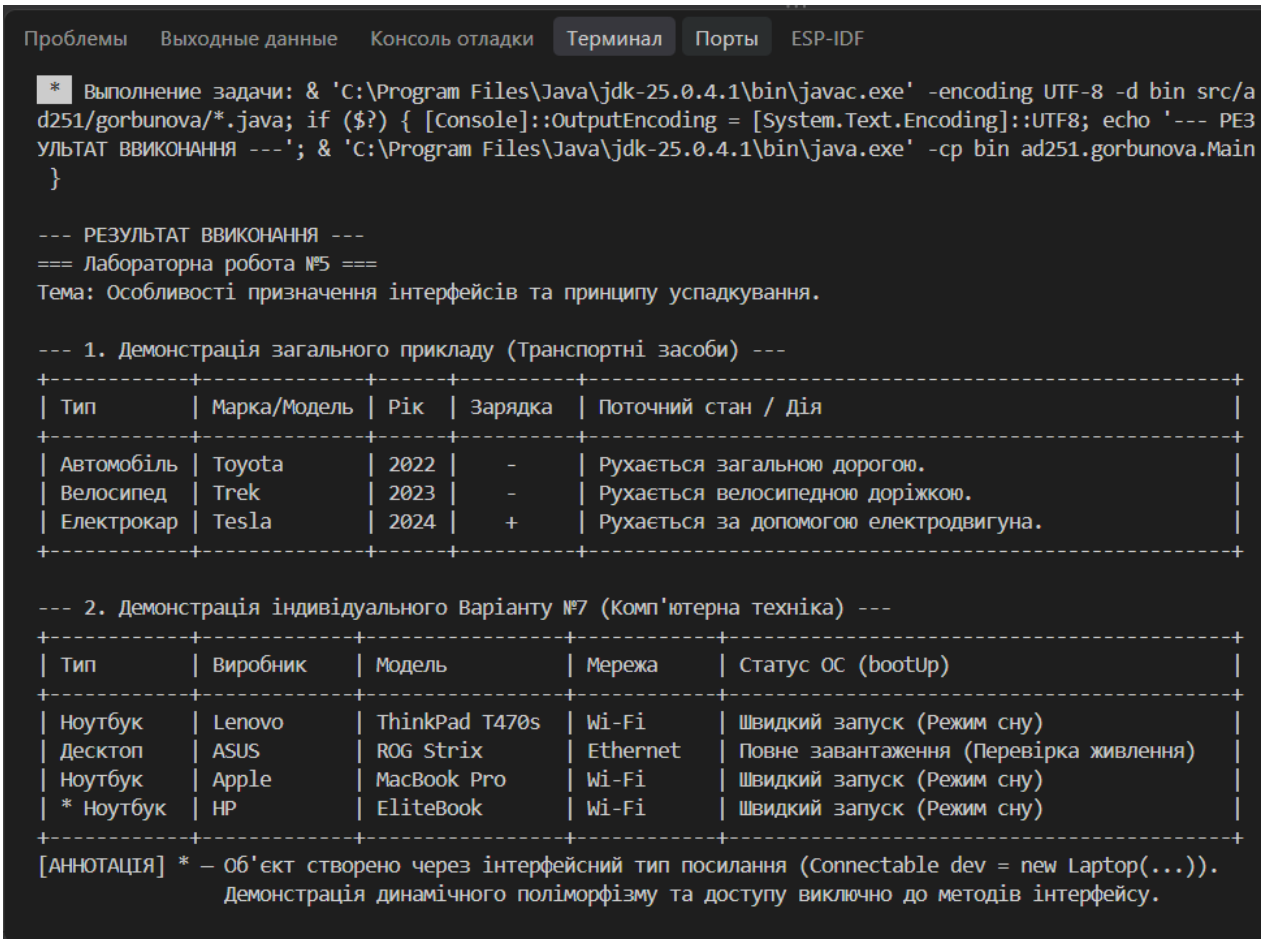



Рисунок 8.1 – Скріншот результатів виконання програми в терміналі VS Code завдання лабораторної роботи №5

Завдяки винесенню логіки форматування в головний клас `Main.java` було реалізовано принцип розділення відповідальності (Separation of Concerns). Дочірні

класи не містять елементів CLI-розмітки, повертаючи виключно інформаційні рядки стану, що дозволяє гнучко масштабувати систему та змінювати формат представлення даних без рефакторингу бізнес-логіки ієрархії.

# 9 ІНФОРМАЦІЙНІ РЕСУРСИ ТА ПОСИЛАННЯ НА ВИХІДНИЙ КОД

Згідно з вимогами до здачі лабораторної роботи, повний проект розробленого вихідного коду додатку, розміщено у віддаленому репозиторії системи контролю версій.

– **Посилання на Git-репозиторій проєкту:**
`https://github.com/nasuehfw/oop-lab5.git`

– **Електронний каталог спільного доступу:**
https://drive.google.com/drive/folders/1oiJNrP62_ZFaXsxSDoPd4y1sXu0Bt5s1?usp=drive_link